File Handling

Introduction

- A computer file is **a resource for recording data on a computer storage device.**
- In Java, with the help of **File** Class, we can work with files.
- This File Class is inside the java.io package.
- **Why File Handling is Required?**
  - File Handling is an integral part of any programming language as file handling enables us to store the output of any particular program in a file and allows us to perform certain operations on it.
  - In simple words, file handling means reading and writing data to a file.

Create a file

```
import java.io.File;
import java.io.IOException;

public class CreateFileExample {
    public static void main(String[] args) {
        File file = new File("example.txt");
        try {
if (file.createNewFile()) {
                System.out.println("File created: " + file.getName());
            } else {
                System.out.println("File already exists.");
            }
        } catch (IOException e) {
            System.out.println("An error occurred.");
            e.printStackTrace();
        }
    }
}
```

File class methods

Method	Type	Description
canRead()	Boolean	Tests whether the file is readable or not
canWrite()	Boolean	Tests whether the file is writable or not
createNewFile()	Boolean	Creates an empty file
delete()	Boolean	Deletes a file
exists()	Boolean	Tests whether the file exists
getName()	String	Returns the name of the file
getAbsolutePath()	String	Returns the absolute pathname of the file
length()	Long	Returns the size of the file in bytes
list()	String[]	Returns an array of the files in the directory
mkdir()	Boolean	Creates a directory

Read a file using Scanner class

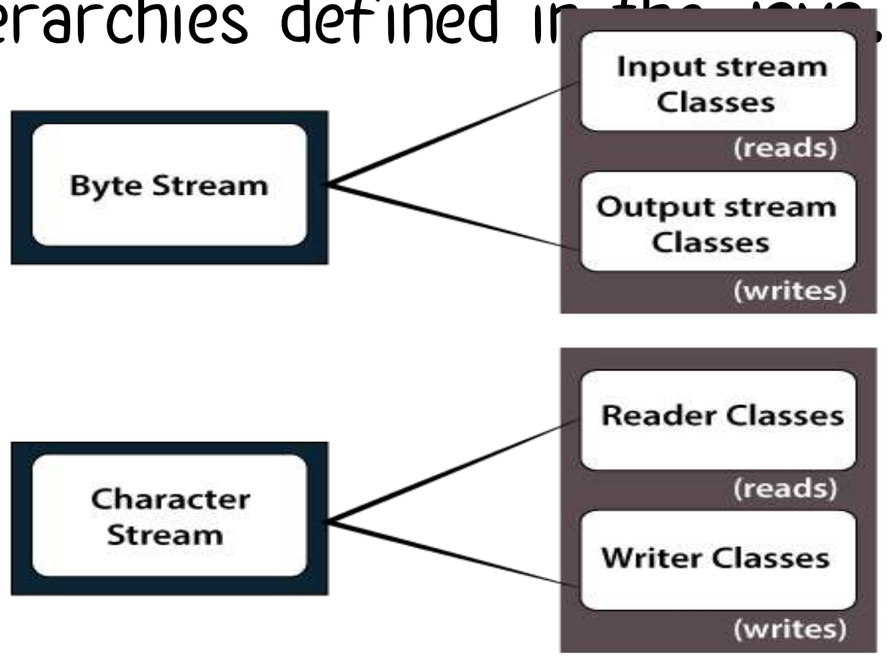
```
import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;
public class GFG {
        public static void main(String[] args){
                try {
                        File Obj = new File("myfile.txt");
                        Scanner Reader = new Scanner(Obj);
                        while (Reader.hasNextLine()) {
                                String data = Reader.nextLine();
                                System.out.println(data);
                        }
                        Reader.close();
                }
                catch (FileNotFoundException e) {
                        System.out.println("An error has occurred.");
                        e.printStackTrace();
                }
        }
}
```

Write into a file

```
import java.io.FileWriter;
import java.io.IOException;
public class GFG {
	public static void main(String[] args)
	{
		try {
			FileWriter Writer= new FileWriter("myfile.txt");
			Writer.write("Files in Java are seriously good!!");
			Writer.close();
			System.out.println("Successfully written.");
		}
		catch (IOException e) {
			System.out.println("An error has occurred.");
			e.printStackTrace();
		}
	}
}
```

Streams in Java

- Java programs perform I/O through streams.
- A *stream* is an abstraction that either produces or consumes information. A stream is linked to a physical device by the Java I/O system.
- Same I/O classes and methods can be applied to different types of devices.
- Java implements streams within class hierarchies defined in the **java.io** package.
- Types of stream
  - Byte stream
  - Character stream



Brief classification of I/O streams

Byte stream and Character stream

- *Byte streams* provide a convenient means for handling input and output of bytes.
- Byte streams are used, for example, when reading or writing binary data.
- *Character streams* provide a convenient means for handling input and output of characters.
- They use Unicode and, therefore, can be internationalized.
- At the lowest level, all I/O is still byte-oriented.
- The character-based streams simply provide a convenient and efficient means for handling characters.

Byte stream

- Byte streams are defined by using two class hierarchies. At the top are two abstract classes: **InputStream** and **OutputStream**.
- Each of these abstract classes has several concrete subclasses that handle the differences among various devices, such as disk files, network connections, and even memory buffers.
- The abstract classes **InputStream** and **OutputStream** define several key methods that the other stream classes implement.
- Two of the most important are **read( )** and **write( )**, which, respectively, read and write bytes of data.

Byte Steam classes in java.io

Stream Class	Meaning
BufferedInputStream	Buffered input stream
BufferedOutputStream	Buffered output stream
ByteArrayInputStream	Input stream that reads from a byte array
ByteArrayOutputStream	Output stream that writes to a byte array
DataInputStream	An input stream that contains methods for reading the Java standard data types
DataOutputStream	An output stream that contains methods for writing the Java standard data types
FileInputStream	Input stream that reads from a file
FileOutputStream	Output stream that writes to a file
FilterInputStream	Implements **InputStream**
FilterOutputStream	Implements **OutputStream**
InputStream	Abstract class that describes stream input
ObjectInputStream	Input stream for objects
ObjectOutputStream	Output stream for objects
OutputStream	Abstract class that describes stream output
PipedInputStream	Input pipe
PipedOutputStream	Output pipe
PrintStream	Output stream that contains **print( )** and **println( )**
PushbackInputStream	Input stream that allows bytes to be returned to the input stream.
SequenceInputStream	Input stream that is a combination of two or more input streams that will be read sequentially, one after the other

Character stream

- Character streams are defined by using two class hierarchies. At the top are two abstract classes: **Reader** and **Writer**.
- The abstract classes **Reader** and **Writer** define several key methods that the other stream classes implement.
- Two of the most important methods are **read()** and **write( )**, which read and write characters of data, respectively.

Character stream class

Stream Class	Meaning
BufferedReader	Buffered input character stream
BufferedWriter	Buffered output character stream
CharArrayReader	Input stream that reads from a character array
CharArrayWriter	Output stream that writes to a character array
FileReader	Input stream that reads from a file
FileWriter	Output stream that writes to a file
FilterReader	Filtered reader
FilterWriter	Filtered writer
InputStreamReader	Input stream that translates bytes to characters
LineNumberReader	Input stream that counts lines
OutputStreamWriter	Output stream that translates characters to bytes
PipedReader	Input pipe
PipedWriter	Output pipe
PrintWriter	Output stream that contains **print( )** and **println( )**
PushbackReader	Input stream that allows characters to be returned to the input stream
Reader	Abstract class that describes character stream input
StringReader	Input stream that reads from a string
StringWriter	Output stream that writes to a string
Writer	Abstract class that describes character stream output

The Predefined Streams

- **java.lang** package defines a class called **System** which contains three stream variables: *in, out,* and *err*.
- System.out refers to the standard output stream. By default, this is the console.
- System.in refers to standard input, which is the keyboard by default.
- System.err refers to the standard error stream, which also is the console by default.
- System.in is an object of type InputStream; System.out and System.err are objects of type PrintStream.These are byte streams.

- **NOTE** : Names of character streams typically end with Reader/Writer & names of byte streams end with InputStream/OutputStream.

Input Stream

- The input stream is used to read data from numerous input devices like the keyboard, network, etc.
- The Java **InputStream** class is the superclass of all input streams.
- InputStream is an abstract class, and because of this, it is not useful by itself. However, its subclasses are used to read data.
- There are several subclasses of the InputStream class, which are as follows:
  - AudioInputStream
  - ByteArrayInputStream
  - FileInputStream
  - FilterInputStream
  - StringBufferInputStream
  - ObjectInputStream
- Creating an InputStream
  - `InputStream obj = new FileInputStream();`

  - ***Note:*** *We can create an input stream from other subclasses as well as InputStream.*

Example: Reading from a File Using FileInputStream

```
import java.io.FileInputStream;
import java.io.InputStream;
import java.io.IOException;

public class FileInputStreamExample {
    public static void main(String[] args) {
        try (InputStream fis = new FileInputStream("test.txt")) {
            int data;
            while ((data = fis.read()) != -1) { // Read byte by byte
                System.out.print((char) data); // Convert byte to character (optional)
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Methods of InputStream

S No.	Method	Description
1	read()	Reads one byte of data from the input stream.
2	read(byte[] array)()	Reads byte from the stream and stores that byte in the specified array.
3	mark()	It marks the position in the input stream until the data has been read.
4	available()	Returns the number of bytes available in the input stream.
5	markSupported()	It checks if the mark() method and the reset() method is supported in the stream.
6	reset()	Returns the control to the point where the mark was set inside the stream.
7	skips()	Skips and removes a particular number of bytes from the input stream.
8	close()	Closes the input stream.

Output Steam

- The output stream is used to write data to numerous output devices like the monitor, file, etc.
- OutputStream is an abstract superclass that represents an output stream. OutputStream is an abstract class and because of this, it is not useful by itself. However, its subclasses are used to write data.
- There are several subclasses of the OutputStream class which are as follows:
  - ByteArrayOutputStream
  - FileOutputStream
  - StringBufferOutputStream
  - ObjectOutputStream
  - DataOutputStream
  - PrintStream
- **Creating an OutputStream**
  - OutputStream obj = new FileOutputStream();
- ***Note:*** *We can create an output stream from other subclasses as well as OutputStream.*

Example: Writing to a File Using FileOutputStream

```
import java.io.FileOutputStream;
import java.io.IOException;

public class FileOutputStreamExample {
    public static void main(String[] args) {
        try (FileOutputStream fos = new FileOutputStream("example.txt")) {
            String content = "Hello, byte stream!";
            fos.write(content.getBytes()); // Convert String to bytes
            System.out.println("Data written to file.");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Methods of OutputStream

S. No.	Method	Description
1.	write()	Writes the specified byte to the output stream.
2.	write(byte[] array)	Writes the bytes which are inside a specific array to the output stream.
3.	close()	Closes the output stream.
4.	flush()	Forces to write all the data present in an output stream to the destination.

File Read using Buffered Streams for Efficiency

```
import java.io.BufferedInputStream;
import java.io.FileInputStream;
import java.io.IOException;

public class BufferedInputStreamExample {
    public static void main(String[] args) {
        try (BufferedInputStream bis = new BufferedInputStream(new FileInputStream("example.txt"))) {
            int data;
            while ((data = bis.read()) != -1) { // Read data in chunks using a buffer
                System.out.print((char) data);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

File Write using Buffered Streams

```
import java.io.FileOutputStream;
import java.io.BufferedOutputStream;
import java.io.IOException;
public class BufferedFileWriteBinary {
    public static void main(String[] args) {
        BufferedOutputStream bos = null;
        try {
                 bos = new BufferedOutputStream(new FileOutputStream("example.dat"));
                byte[] data = {65, 66, 67, 68, 69};       // ASCII values for 'A', 'B', 'C', 'D', 'E'
                bos.write(data);
               System.out.println("Data written to file successfully.");
        } catch (IOException e) {
            e.printStackTrace();
        } finally {
            try {
                if (bos != null) {
                    bos.close();  // Close the BufferedOutputStream
                }
            } catch (IOException e) {
                e.printStackTrace();
            }
        }
    }
```

Delete a file

```
import java.io.File;
public class GFG {
	public static void main(String[] args){
		File Obj = new File("myfile.txt");
		if (Obj.delete()) {
			System.out.println("The deleted file is : "+ Obj.getName());
		}
		else {
			System.out.println("Failed in deleting the file.");
		}
	}
}
```

Filter stream

- In Java, **filter streams** are streams that process data read from or written to another stream.
- They are used to modify or filter the data in some way while it is being read or written.
- A filter stream can wrap an existing stream to provide additional functionality, such as buffering, data conversion, or compression.
- Common Filter Stream ClassesBuffered Streams:
  - BufferedInputStream, BufferedOutputStream, BufferedReader,BufferedWriter.
  - Data Streams: DataInputStream, DataOutputStream.Object
  - Streams: ObjectInputStream, ObjectOutputStream.
  - Others: PrintStream, PushbackInputStream.

Using DataOutputStream and DataInputStream

- This example demonstrates writing primitive data types to a file and reading them back.

```
import java.io.DataInputStream;
import java.io.DataOutputStream;
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;

public class DataStreamExample {
    public static void main(String[] args) {
        // Writing data to a file
        try (DataOutputStream dos = new
DataOutputStream(new FileOutputStream("data.bin"))) {
            dos.writeInt(42);           // Write an integer
            dos.writeDouble(3.14);      // Write a double
            dos.writeUTF("Hello!");     // Write a UTF-8
encoded string
        } catch (IOException e) {
            System.out.println("Error writing data: " +
e.getMessage());
        }
        // Reading data from the file
        try (DataInputStream dis = new
DataInputStream(new FileInputStream("data.bin"))) {
            int intValue = dis.readInt();
            double doubleValue = dis.readDouble();
            String stringValue = dis.readUTF();

            System.out.println("Read Integer: " + intValue);
            System.out.println("Read Double: " + doubleValue);
            System.out.println("Read String: " + stringValue);
        } catch (IOException e) {
            System.out.println("Error reading data: " +
e.getMessage());
        }
    }
}
```

Using ObjectOutputStream and ObjectInputStream

- This example demonstrates writing and reading objects to/from a file.

```
import java.io.*;
class Person implements Serializable {
    private static final long serialVersionUID = 1L;
    String name;
    int age;
    Person(String name, int age) {
        this.name = name;
        this.age = age;
    }
    @Override
    public String toString() {
        return "Person{name='" + name + "', age=" + age + "}";
    }
}
public class ObjectStreamExample {
    public static void main(String[] args) {
        // Writing an object to a file
        try (ObjectOutputStream oos = new
ObjectOutputStream(new FileOutputStream("person.obj"))) {
            Person person = new Person("Alice", 30);
            oos.writeObject(person);
        } catch (IOException e) {
            System.out.println("Error writing object: " +
e.getMessage());
        }

        // Reading an object from the file
        try (ObjectInputStream ois = new
ObjectInputStream(new FileInputStream("person.obj"))) {
            Person person = (Person) ois.readObject();
            System.out.println("Read Object: " + person);
        } catch (IOException | ClassNotFoundException e) {
            System.out.println("Error reading object: " +
e.getMessage());
        }
    }
}
```

PrintStream

- In Java, PrintStream is a subclass of OutputStream that is used to write formatted text (such as strings, numbers, or objects) to an output destination like a file, console, or network socket.
- It is part of the java.io package and provides methods to output data in a human-readable format.
- Automatic Flushing: If autoFlush is enabled, the stream flushes automatically whenever a newline character (\n) is written, or one of the print/println methods is called.
- No I/O Exceptions:Unlike other streams, PrintStream does not throw IOException during operations. Instead, errors are internally handled, and you can check the error status using the checkError() method.
- Convenient Printing: Provides easy-to-use methods like print() and println() for outputting various data types, including primitives, objects, and strings.
- Encapsulation: Wraps around another output stream, such as FileOutputStream, to make it easier to write human-readable text.

PrintStream

- Common Constructors
  - PrintStream(OutputStream out): Creates a new print stream.
  - PrintStream(OutputStream out, boolean autoFlush): Enables/disables automatic flushing.
  - PrintStream(String fileName): Writes data to a file with the given name.
  - PrintStream(File file): Writes data to a specified file object

Key Methods of PrintStream

1.Output Methods:

- print(): Prints data without a newline.
- println(): Prints data followed by a newline.

2.Formatting:

- printf(String format, Object... args): Writes formatted data, similar to System.out.printf().

3.Error Handling:

- checkError(): Returns true if the stream has encountered an error.

4.Flushing and Closing:

- flush(): Forces any buffered output to be written.
- close(): Closes the stream and releases resources.

Example 1: Writing to the Console

- By default, System.out is a PrintStream, so you've likely used it without realizing it:

```
public class PrintStreamExample {
    public static void main(String[] args) {
        PrintStream ps = System.out; // System.out is a PrintStream
        ps.println("Hello, PrintStream!"); // Prints with a newline
        ps.print("This is ");                // Prints without a newline
        ps.print("on the same line.");      // Continues on the same line
    }
}
```

Example 2: Writing to a File

- You can wrap a FileOutputStream in a PrintStream to write formatted text to a file.

```
import java.io.FileOutputStream;
import java.io.PrintStream;
import java.io.IOException;
public class PrintStreamToFile {
    public static void main(String[] args) {
        try (PrintStream ps = new PrintStream(new FileOutputStream("output.txt"))) {
            ps.println("This is a line of text.");
            ps.println(123); // Writes an integer
            ps.printf("Formatted number: %.2f%n", 45.6789); // Writes a formatted number
        } catch (IOException e) {
            System.out.println("An error occurred: " + e.getMessage());
        }
    }
}
```

Example 3: Error Handling with `checkError()`

```
import java.io.PrintStream;
public class CheckErrorExample {
    public static void main(String[] args) {
        PrintStream ps = null;
        try {
            ps = new PrintStream("invalid/path/output.txt");
            ps.println("This will cause an error!");
        } catch (Exception e) {
            System.out.println("Caught an exception: " + e.getMessage());
        } finally {
            if (ps != null && ps.checkError()) {
                System.out.println("An error occurred while writing to the stream.");
            }
            if (ps != null) {
                ps.close();
            }
        }
    }
}
```

Reader Class

- The Reader class is the superclass for all character-based input streams. It provides methods to read characters, arrays of characters, or lines from a source.
- Key Subclasses:
  - BufferedReader: For efficient reading using a buffer.
  - InputStreamReader: Converts byte streams to character streams.
  - FileReader: For reading files as characters.
- Common Methods in Reader:
  - int read(): Reads a single character.
  - int read(char[] cbuf): Reads characters into an array.
  - void close(): Closes the reader.

Writer Class

- The Writer class is the superclass for all character-based output streams. It provides methods to write characters, arrays of characters, or strings to a destination.
- Key Subclasses:
  - BufferedWriter: For efficient writing using a buffer.
  - OutputStreamWriter: Converts character streams to byte streams.
  - FileWriter: For writing files as characters.
- Common Methods in Reader:
  - void write(int c): Writes a single character.
  - void write(char[] cbuf): Writes an array of characters.
  - void write(String str): Writes a string.
  - void flush(): Flushes the stream.
  - void close(): Closes the writer

Example 1: Reading from a File using Reader

```
import java.io.FileReader;
import java.io.IOException;

public class ReaderExample {
    public static void main(String[] args) {
        try (FileReader reader = new FileReader("example.txt")) {
            int character;
            while ((character = reader.read()) != -1) {
                System.out.print((char) character);
            }
        } catch (IOException e) {
            System.out.println("An error occurred: " + e.getMessage());
        }
    }
}
```

Example 2: Writing to a File using Writer

```
import java.io.FileWriter;
import java.io.IOException;

public class WriterExample {
    public static void main(String[] args) {
        try (FileWriter writer = new FileWriter("output.txt")) {
            writer.write("Hello, World!\n");
            writer.write("This is an example of using Writer.");
        } catch (IOException e) {
            System.out.println("An error occurred: " + e.getMessage());
        }
    }
}
```

Example 3: BufferedReader and BufferedWriter

```
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
public class BufferedReaderWriterExample {
    public static void main(String[] args) {
        // Write to a file
        try (BufferedWriter writer = new
BufferedWriter(new FileWriter("buffered_output.txt")))
{
            writer.write("This is the first line.");
            writer.newLine(); // Writes a newline
            writer.write("This is the second line.");
        } catch (IOException e) {
            System.out.println("Error writing to the
file: " + e.getMessage());
        }
 // Read from the file
        try (BufferedReader reader = new
BufferedReader(new FileReader("buffered_output.txt")))
{
            String line;
            while ((line = reader.readLine()) != null) { //
Reads line by line
                System.out.println(line);
            }
        } catch (IOException e) {
            System.out.println("Error reading the file: "
+ e.getMessage());
        }
    }
}
```

Random access files

- In Java, **Random Access Files** provide a way to read from and write to a file at any position. Unlike sequential access files, where data is read or written in order, random access files allow you to jump to a specific location in the file and perform read/write operations.
- The class that supports random access file operations is RandomAccessFile, part of the java.io package.
- Key Features of RandomAccessFile
- Random Access:
  - You can move the file pointer to any location in the file using the seek() method.
  - It allows non-sequential (random) reading and writing.
- Dual Modes:
  - "r": Open the file in read-only mode.
  - "rw": Open the file in read/write mode.
- Supports Both Reading and Writing:
  - Provides methods to read (readInt(), readLine(), etc.) and write (writeInt(), writeUTF(), etc.) various data types.

Random access files

- Constructors
  - RandomAccessFile(String name, String mode): Opens the file with the specified name and mode.
  - RandomAccessFile(File file, String mode): Opens the specified File object.
- Common methods:

Method	Description
read()	Reads a single byte of data.
read(byte[] b)	Reads a sequence of bytes into an array.
write(int b)	Writes a single byte to the file.
write(byte[] b)	Writes an array of bytes to the file.
seek(long pos)	Moves the file pointer to the specified position.
getFilePointer()	Returns the current position of the file pointer.
length()	Returns the length of the file in bytes.
setLength(long newLength)	Sets the length of the file (truncates or extends).
close()	Closes the file.

Example 1: Writing and Reading Using RandomAccessFile

```
import java.io.RandomAccessFile;

public class RandomAccessFileExample {
    public static void main(String[] args) {
        try (RandomAccessFile raf = new
RandomAccessFile("example.txt", "rw")) {
            // Writing data to the file
            raf.writeUTF("Hello, World!");  // Write a
string
            raf.writeInt(42);               // Write an
integer
            raf.writeDouble(3.14);          // Write a
double

            // Seeking to the beginning of the file
            raf.seek(0);

             // Reading data from the file
            String message = raf.readUTF(); // Read
the string
            int number = raf.readInt();     // Read the
integer
            double pi = raf.readDouble();   // Read the
double

            System.out.println("Message: " + message);
            System.out.println("Number: " + number);
            System.out.println("Value of Pi: " + pi);
        } catch (Exception e) {
            System.out.println("An error occurred: " +
e.getMessage());
        }
    }
}
```

Example 2: Modifying a Specific Part of a File

```
import java.io.RandomAccessFile;

public class ModifyFileExample {
    public static void main(String[] args) {
        try (RandomAccessFile raf = new
RandomAccessFile("modify_example.txt", "rw")) {
            // Writing initial data
            raf.writeUTF("Initial Data");
            raf.writeInt(100);
            raf.writeUTF("Some other data");

            // Modify the integer at a specific
position
            raf.seek(12); // Seek to the position where
the integer is stored
            raf.writeInt(200); // Overwrite the integer

// Read the file to verify the update
            raf.seek(0); // Go back to the beginning
            System.out.println("String 1: " +
raf.readUTF());
            System.out.println("Updated Integer: " +
raf.readInt());
            System.out.println("String 2: " +
raf.readUTF());
        } catch (Exception e) {
            System.out.println("An error occurred: " +
e.getMessage());
        }
    }
}
```